

Exercise 6:

```
syms f(x,y);
f(x,y) = x^2-3*y^2+x*y^3;
symbolic = double(int(int(f,y,0,4),-2,2));
Trap = d2_trapezoidal(@integrand,-2,2,0,4,100);
built_int = integral2(@integrand,-2,2,0,4);
Simpson = simpson_2d(@integrand,-2,2,0,4,100);
fprintf("Analytical result = %f \n" + ...
    "Built in integral function result = %f \n" + ...
    "Trapezoidal result = %f \n" + ...
    "Simpson Result = %f \n",symbolic,built_int,Trap,Simpson)
```

```
Analytical result = -234.666667
Built in integral function result = -234.666667
Trapezoidal result = -234.675200
Simpson Result = -234.666667
```

We can see that all the results are really close and the built in integral function gave the same result as the analytical solution. The trapezoidal numerical function has a percentage error less than 0% when compare to the analytical and built in integral results. Similarly, the Simpson method also has less than 0% percentage error when compared the other two methods. However, if we want to be really close to the analytical function (and hence the integral function), is better to use the Simpson method.

```
100*abs(Trap - built_int)/built_int ;
100*abs(Simpson- built_int)/built_int;
```

```
function [z] = integrand(x,y)
z = x.^2-3.*y.^2+x.*y.^3;
end

function [I] = d2_trapezoidal(f,ax,bx,ay,by,n)
I = 0;
h=(by-ay)/n;% for the y limits
for i = 0:n
    y = ay + i*h;
    if (i==0 || i==n)
        I = I + segment_trapezoidal(f,y,ax,bx,n);
    else
        I = I + 2*segment_trapezoidal(f,y,ax,bx,n);
    end
end

I = I*h/2;
end

function [I] = segment_trapezoidal(f,y,a,b,n)
I=0; % initial value of integral
h=(b-a)/n;
for i=0:n
    x= a + i*h; % goin from a to b
```

```

if (i==0 || i==n)
    I = I + f(x,y);
else % if is not 0 or n
    I = I + 2*f(x,y);
end
end
I = I*h/2;
end
% simpson:
function [I] = simpson_2d(f,ax,bx,ay,by,n) % "calculate" inner int
    if (mod(n,2)==0)
        I=0;
        h = (by-ay)/n;
        for i =0:n
            y = ay + i*h;
            if (i==0 || i==n)
                I = I + simpson_one_third(f,y,ax,bx,n);
            elseif (mod(i,2)==1)
                I = I + 4*simpson_one_third(f,y,ax,bx,n);
            else
                I = I + 2*simpson_one_third(f,y,ax,bx,n);
            end
        end
        I = I*h/3;
    else
        disp("n has to be even")
    end
end

function [I] = simpson_one_third(f,y,a,b,n) % this calculates outer integral
if (mod(n,2)==0)
    I=0;
    h = (b-a)/n;
    for i =0:n
        x = a + i*h;
        if (i==0 || i==n)
            I = I + f(x,y);
        elseif (mod(i,2)==1)
            I = I + 4*f(x,y);
        else
            I = I + 2*f(x,y);
        end
    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

```